project 3 - Threads and Synchronization

Mika Yamin - 325115996
Bar Hana Yehezkel - 322723206

Q1. Synchronization

Q1 - A.

Yes, we can technically build it, but it is a very bad design.
Since we are only allowed to use a Mutex, we have to use it just to protect our counter. For the waiting part, we are forced to use an endless loop.

```
using System;
using System.Threading;

class MySemaphore
{
   private int currentCount;
   private int maxCount;
   private Mutex countMutex;

   // Constructor
   public MySemaphore(int starting, int max)
   {
      this.currentCount = starting;
      this.maxCount = max;
      // The only tool we are allowed to use
      this.countMutex = new Mutex();
   }

   // Wait method
   public bool WaitOne()
   {
      // We are forced to use an endless loop (Busy Waiting)
      while (true)
      {
         countMutex.WaitOne(); // Lock to check the counter safely

         if (currentCount > 0)
         {
            currentCount--; // Take one free spot
            countMutex.ReleaseMutex(); // Unlock
            return true;
         }

         // If no spots are free, unlock immediately so other threads can enter and release spots
         countMutex.ReleaseMutex();

         // Sleep for a tiny bit so the CPU doesn't crash from working too hard
         Thread.Sleep(1);
      }
   }

   // Release method
```

```
    public bool Release(int num = 1)
    {
        countMutex.WaitOne(); // Lock to add spots safely

        if (currentCount + num > maxCount)
        {
            countMutex.ReleaseMutex();
            throw new Exception("Semaphore is full.");
        }

        currentCount += num; // Add spots back

        countMutex.ReleaseMutex(); // Unlock
        return true;
    }
}
```

Why is this a bad idea:

1. Wasting CPU power (Busy Waiting): A real synchronization tool puts a waiting thread to sleep so it doesn't waste the computer's power. Because we only have a Mutex, our waiting threads get stuck in an endless while(true) loop. This is called "busy waiting". It means the CPU spins and works extremely hard doing absolutely nothing, just asking "is there a free spot yet?" over and over again.
2. Mutex Ownership: A very strict rule of a Mutex is that only the specific thread that locked it is allowed to unlock it. In a real Semaphore, it is completely normal for one thread to wait for a spot, and a totally different thread to release that spot and wake it up. Because a Mutex only belongs to its owner, we cannot use it to make one thread block another thread. We can only use it for a split second just to protect our currentCount number from getting mixed up.

Q1 - B.

Yes, we can absolutely use semaphores to force this exact order.
To do this, we use semaphores like "wait signals" or traffic lights. If we start a semaphore at 0, any thread that tries to pass it will immediately fall asleep and wait until another thread adds to it (signals).

We will need two semaphores, both starting at 0:

- sem_S1_done (starts at 0)
- sem_S2_done (starts at 0)

Here is how we set up the three threads in C#:
Thread 1:

```
S1;
sem_S1_done.Release(); // Signals that S1 is finished (adds 1)
```

Thread 2:

```
sem_S1_done.WaitOne(); // Goes to sleep until Thread 1 is done
S2;
sem_S2_done.Release(); // Signals that S2 is finished (adds 1)
```

Thread 3:
sem_S2_done.WaitOne(); // Goes to sleep until Thread 2 is done
S3;

Why this works perfectly:

- If Thread 3 tries to run first, it hits WaitOne(). Since the semaphore is at 0, Thread 3 is blocked and goes to sleep.
- If Thread 2 tries to run, it also hits a WaitOne() that is at 0, so it goes to sleep too.
- Only Thread 1 can actually do its work. It runs S1, and then calls Release() on the first semaphore.
- This wakes up Thread 2. Thread 2 now runs S2, and then calls Release() on the second semaphore.
- Finally, this wakes up Thread 3, which can now safely run S3.

This creates a perfect step-by-step chain reaction (S1 -> S2 -> S3), no matter which thread the computer tries to start first.

Q1 - C.

The fundamental difference between the two algorithms lies in their approach to the turn variable: Peterson's solution relies on preemptive yielding (giving priority away before waiting), whereas Dekker's solution uses it to enforce strict right-of-way only when a conflict occurs.

Feature	**Dekker's solution**	**Peterson's solution**
How it works (Who goes next)	Uses two "I want in" flags and a 'turn' sign. The turn is only used as a tie-breaker if both threads try to enter at the exact same time.	A thread raises its flag to say "I want in", but politely gives the 'turn' to the other thread right away.
Code Complexity	More complicated. It uses loops inside of loops. The thread that doesn't have the turn has to take its flag down and wait.	Short, simple, and neat. It uses just one basic waiting loop with a simple condition.
Hardware Requirements	A pure software solution using shared memory. It doesn't need any special hardware tricks, making it easy to run on almost any basic computer.	Also a pure software solution using shared memory (relies on basic read and write actions).
What it guarantees	Makes sure they don't crash into each other (Mutual exclusion), the system doesn't freeze (No deadlock), and no thread is ignored forever (No starvation).	Makes sure they don't crash into each other (Mutual exclusion), the system keeps moving (Progress), and nobody waits forever (Bounded waiting).
Advantages	Extremely fast to get in and out of the shared resource if there is no actual competition between the threads at that moment.	Very easy to read, understand, and write in code compared to Dekker.

Disadvantages	1. Only works for exactly two threads. 2. Uses "busy waiting" (spins in an endless loop and wastes CPU power). 3. It crashes on modern multi-core processors that mix up the order of instructions, unless you manually add "memory barriers" to the code.	1. Designed only for two threads; trying to make it work for many threads is messy and impractical. 2. Also suffers from "busy waiting" which wastes CPU power. 3. Just like Dekker, it fails on modern multi-core processors without manual memory barriers.

Q2. Thread-Safe Binary Tree

We based our synchronization strategy for the ThreadSafeBinaryTree on the Readers-Writers paradigm, which allowed us to provide high concurrency for read operations while ensuring strict mutual exclusion for write operations.

1. Synchronization Variables: To implement this paradigm, we used the following variables:

- read_count (int): We use this to track the number of active readers currently accessing the tree.
- mutex (Mutex): We implemented a standard mutual exclusion lock solely to protect the read_count variable from race conditions when multiple readers enter or exit simultaneously.
- rw_mutex (SemaphoreSlim): We chose this as our primary lock to protect the critical section (the binary tree structure itself).

2. Crucial Design Choice - Overcoming Thread Affinity: A critical design decision we made was using SemaphoreSlim(1, 1) for the rw_mutex instead of a standard C# Mutex. In the Readers-Writers protocol, the first reader to enter acquires the lock to block writers, and the last reader to leave releases it. In our highly concurrent stress tests, the first and last readers are often completely different threads. Since a standard Mutex enforces Thread Affinity (only the acquiring thread can release it), attempting to release it from a different thread would throw an exception and cause a deadlock. By choosing SemaphoreSlim, which acts as a binary semaphore and does not enforce thread ownership, we created a safe and robust solution for this protocol.

3. Readers (Search and PrintSorted): Because the Search and PrintSorted methods do not modify the tree, we defined them as Readers. When a reader enters, we lock the mutex and increment the read_count. If it is the first reader (read_count == 1), we acquire the rw_mutex to block any incoming writers. We allow other readers to continue entering the tree concurrently without waiting for the rw_mutex. Upon exiting, we decrement the read_count, and if it is the last reader (read_count == 0), we release the rw_mutex, allowing writers to access the tree again.

4. Writers (Add and Delete): We defined the Add and Delete methods as Writers since they modify the tree structure or reference counts. We require a writer to acquire the rw_mutex exclusively before executing. This ensures that while we are adding, modifying, or physically removing a node, no other writers or readers can access the tree, preventing any data corruption.

5. Robustness and Edge Cases:

- Absolute Case Sensitivity: To fully comply with the assignment's case-sensitive requirement, we used string.CompareOrdinal() instead of string.Compare(). This ensures our alphabetical sorting and comparisons are strictly based on absolute ASCII values, unaffected by local OS culture settings.

- Null Safety: We added a preliminary check in all methods to throw an ArgumentNullException if a null value is passed, preventing runtime crashes before acquiring any locks. Furthermore, we carefully wrapped all our synchronization locks in try...finally blocks to guarantee that we always release the locks even if an unexpected exception occurs.

Q3. A. Implement Sharable Spreadsheet object

Operation Groups:

- Group 1 (Read-only): getCell, searchString, searchInRow, searchInCol, searchInRange, findAll, getSize, save.
- Group 2 (Structural/Size changes): addRow, addCol, load.
- Group 3 (Read/Write without resizing): setCell, setAll, exchangeRows, exchangeCols.

We started by dividing the operations into these groups, which helped us understand exactly what our architecture needed to support.

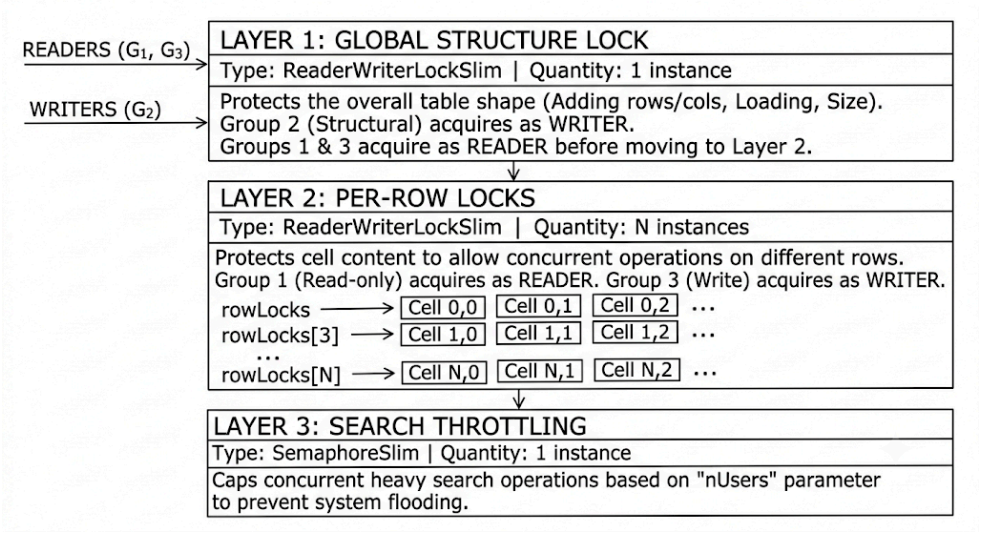
Synchronization Architecture (3 Layers):

1. Layer 1 (structureLock - ReaderWriterLockSlim): Global lock protecting the table's shape. Group 2 operations acquire it as a Writer (exclusive). All other operations acquire it as a Reader to prevent structural changes mid-operation.
2. Layer 2 (Per-row ReaderWriterLockSlim): After acquiring Layer 1, Group 1 operations lock the required row(s) as a Reader (allowing concurrent reads), while Group 3 operations lock them as a Writer (exclusive per row).
3. Layer 3 (searchSemaphore - SemaphoreSlim): Throttles heavy concurrent search operations (based on nUsers) to prevent system flooding, handling resource management rather than data correctness.

Design Decisions & Problem Solving:

- Per-Row vs. Per-Cell Locking: We debated extensively between locking at the level of a single cell versus a row for read/write operations due to the specific advantages and disadvantages of each approach. Per-cell locking causes massive memory waste in large tables and high overhead during row scans, nullifying concurrency benefits. Ultimately, we concluded that row-level locking provides the optimal, most efficient balance.
- Deadlock Avoidance: Strict lock ordering is enforced: the global lock is always acquired before any row lock. Multi-row operations (e.g., exchangeRows) acquire row locks in ascending index order. All locks are released inside try/finally blocks.
- Error Handling: Descriptive exceptions are thrown for any invalid parameters, out-of-bounds indices, or missing search targets.
-
- Lock Types: Two layers of reader-writer locks and a single counting semaphore.
- Lock Count: 1 global lock + N row locks (N = current row count) + 1 semaphore

  Diagram of our design:



Q3.B.Multiple users simulator

```
User [100] at 15:47:12: findAll "R7C21" (case-sensitive) returned 0 cell(s).
User [103] at 15:47:12: replaced all "R13C6" cells with "upd960".
User [102] at 15:47:12: added a new row after row 22.
User [106] at 15:47:12: operation skipped (String "R24C1" was not found in the s
preadsheet.)
User [105] at 15:47:12: read cell [18,0], got "".
User [104] at 15:47:12: exchanged column 21 with column 3.
User [101] at 15:47:12: replaced all "R23C15" cells with "upd892".
User [100] at 15:47:12: added a new row after row 18.
User [107] at 15:47:12: operation skipped (String "R26C6" was not found in the s
preadsheet.)
User [103] at 15:47:12: read cell [13,10], got "R18C8".
User [106] at 15:47:12: exchanged row 11 with row 15.
User [102] at 15:47:12: added a new column after column 7.
User [101] at 15:47:12: operation skipped (String "R29C27" was not found in row
29.)
User [107] at 15:47:12: searched "R16C18", found at cell [27,7].
User [105] at 15:47:12: exchanged column 11 with column 2.
User [100] at 15:47:12: replaced all "R14C4" cells with "upd177".
User [104] at 15:47:12: operation skipped (String "R5C25" was not found in the s
preadsheet.)
User [102] at 15:47:12: wrote "val392" into cell [24,1].
User [106] at 15:47:12: findAll "R19C11" (case-sensitive) returned 1 cell(s).
User [103] at 15:47:12: operation skipped (String "R1C22" was not found in the s
preadsheet.)

---- Simulation finished ----

---- Spreadsheet AFTER simulation ----
R0C3      R0C1      R0C16     -         R0C4      -         R0C5      R0C18
...
R15C3     R15C1     R15C16    -         R15C4     -         R15C5     R15C18
...
-         -         -         -         -         -         -         -
...
R14C3     R14C1     R14C16    -         upd177    -         R14C5     R14C18
...
R3C3      R3C1      R3C16     -         R3C4      -         R3C5      R3C18
...
R5C3      R5C1      R5C16     -         R5C4      -         R5C5      R5C18
```

link to video - q3bvideo.mp4

Q3.C.Spreadsheet GUI application

How We Built the UI:

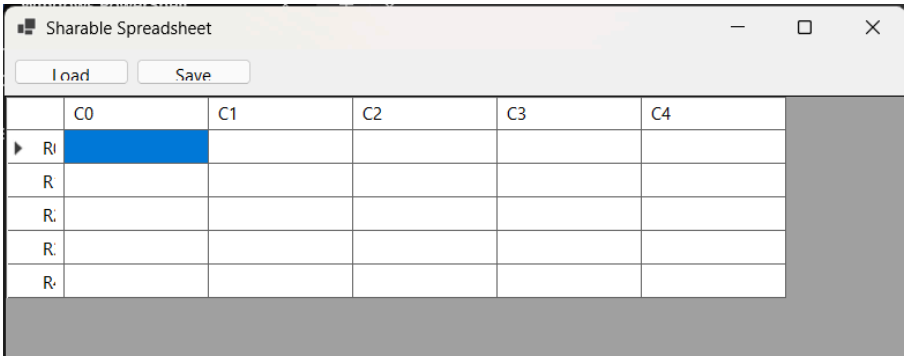
To make our spreadsheet easy to use, we created a Windows Forms app called SpreadsheetApp. We used a DataGridView control to display the table. After watching the recommended YouTube tutorial, we decided the safest way to edit data is outside the grid. When a user clicks a cell, its content appears in a TextBox at the top of the screen. They edit the text there and simply click an "Update" button to save it.

The 3 Main Operations As requested, the app strictly supports only three operations:

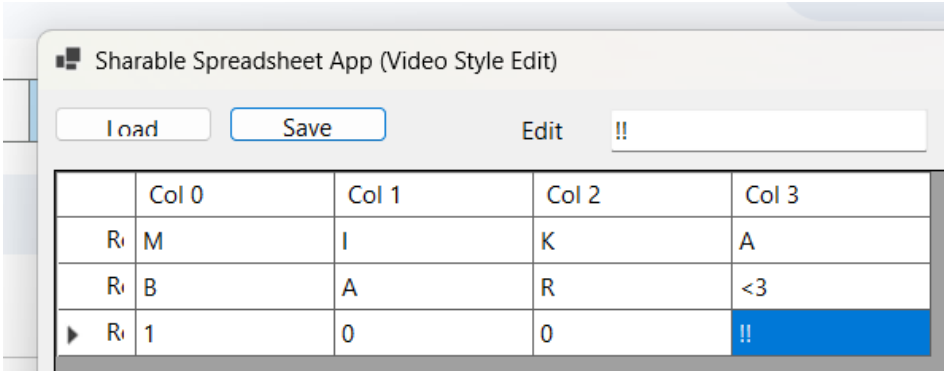
- Edit: Clicking the "Update" button calls our thread-safe setCell method and instantly updates the grid on the screen.
- Load: We used an OpenFileDialog to let the user pick a .txt file. It calls our load method and then rebuilds the table to show the new data.
- Save: We used a SaveFileDialog. It calls our save method to easily store the current table into a text file.

Keeping it Crash-Free We wanted the app to be stable and user-friendly. So, we wrapped all the operations in try...catch blocks. If someone tries to load a bad file or do something invalid, the app won't crash. Instead, it just shows a simple error pop-up (MessageBox).

Before Load and edit



After load and edit



link to video - q3cvideo.mp4